



CMPT 3830: Machine Learning Work Integrated Learning-1

Project Report: Phase 2
Project Title: ALPACA

In collaboration with



Submitted By:

Name	ID	Email
Manpreet Kaur Loyal	3132175	mloyal@norquest.ca
Kirandeep Kaur	3122707	Kkaur707@norquest.ca
Ramon Rabusa	3102642	rrabusa@norquest.ca



Table of Contents:

A table of content is required with section numbers, names with correct page numbers. An example is shown below.

Contents

List of Figures:	3
List of Tables:	4
1. Project Phase:.....	5
2. Team Members' Name with specific roles.....	5
3. Reporting Period	5
4. Project Overview:	6
5. Machine Learning Model:	6
5.1 Model Optimization:	6
5.2 Advanced Analysis:	11
6. Challenges Encountered:.....	13
7. Stakeholder Engagement:	13
8. Lessons Learned:	14
9. Future Recommendations:	14
10. Impact on the Community:	14
11. Project Conclusion:	15
12. Acknowledgments:.....	15
13. Appendices:	15
14. References	18

**List of Figures:**

Figure 1: Reporting Period.....	6
Figure 2: loyalty_treatment Encoding code snippet.....	7
Figure 3: scan_category Encoding code snippet	7
Figure 4: make Encoding code snippet	8
Figure 5: model Encoding code snippet	8
Figure 6: Customer aggregation code snippet	9
Figure 7: Clustering feature code snippet.....	10
Figure 8: Sprint 4	16
Figure 9: Sprint 5	17
Figure 10: Sprint 6.....	17
Figure 11: Sprint 7.....	17



List of Tables:

Table 1: Algorithm Comparison.....	11
Table 2: Cluster Profiling.....	11
Table 3: Make-Specific Clustering Results.....	12



1. Project Phase:

Phase 2 was all about moving past the data cleaning and exploration we did in Phase 1 and getting into the machine learning side of things. We tested out a few different clustering algorithms like KMeans, GMM, and DBSCAN to see which one gave us the best customer segments, and we compared them using silhouette scores. On top of that, we did a dealer-level performance comparison to see how loyalty program usage varies across GO Auto's locations.

One thing that ended up taking more time than we expected was figuring out the right way to encode our categorical features, since getting that wrong threw off the whole clustering. We started with the cleaned dataset from Phase 1 (about 240,000 service records) and aggregated everything down to the customer level, which gave us roughly 53,000 unique VINs to work with. From there, we built out the clustering pipeline with the goal of finding customer groups that actually mean something to GO Auto, not just statistical clusters, but segments they could realistically act on to improve their loyalty program.

2. Team Members' Name with specific roles

Manpreet Kaur Loyal – Project Coordinator: Responsible for project coordination, problem understanding, feature engineering, visualizations and linking machine learning results to business insights. Also contributes to data analysis, implementation, result interpretation, and documentation.

Kirandeep Kaur – Data Preprocessing and Visualization: Responsible for dataset exploration, data preprocessing, and feature preparation. Also contributes to data analysis, machine learning implementation, creation of visualizations, and documentation of findings for reports and presentations.

Ramon Rabusa – ML Implementation and Statistical Analysis: Responsible for implementing clustering algorithms, ML model development, optimization, and validation. Also handles statistical analysis, model performance evaluation, code quality assurance, and GitHub repository management.

3. Reporting Period:

February 2026 to March 2026



Date	Milestone
Feb 10, 2026	Demo 1 – EDA and Data Exploration
Feb 27, 2026	Phase 1 Report submitted
Mar 10, 2026	Demo 2 – ML Modelling and Optimization
Mar 20, 2026	Phase 2 Report submitted
Apr 7, 2026	Final Demo – Full project with Power BI dashboard

Figure 1: Reporting Period

4. Project Overview:

GO Auto operates seven dealerships across Edmonton and runs a loyalty card program for service customers. The question we set out to answer is pretty straightforward: "Does the loyalty program actually make a difference?" Specifically, we wanted to know if dealerships with higher loyalty card usage see better retention, higher revenue per visit, and more consistent service activity compared to locations where customers barely use the card.

This falls under Problem Statement #5 from the project brief: Loyalty-Driven Dealer Performance using clustering and comparative analysis.

Our approach had a few main steps. First, we took the cleaned Phase 1 dataset and aggregated it from the transaction level to the customer level, since clustering needs one row per customer, not one row per service visit. We created features like total visits, average cost per visit, average distance traveled, loyalty ratio (what fraction of visits involved a card scan), and encoded categorical variables like scan_category and loyalty_treatment.

Then we ran KMeans clustering with different values of k, compared it against GMM and DBSCAN, and picked the model that gave us the best silhouette score. After that, we profiled each cluster to understand what kind of customers fall into each group, and we looked at how those segments are distributed across dealerships.

5. Machine Learning Model:

We used unsupervised learning for the main segmentation task because the customer segments are not predefined, where there are no labels in the data telling us which group a customer belongs to. The idea is to let the algorithm find natural groupings based on behavioral patterns.

5.1 Model Optimization:

Before we could do any modelling, we had to get all the features into numeric form. This was actually one of the more involved parts of the project. Here is what we did:

- Ordinal encoding for loyalty_treatment (Non-Loyal=0, Partially Engaged=1, Likely Loyal=2) and scan_category (Never Scanned=0, Intermittent Scanner=1, Always Scanned=2). These have a clear natural order so ordinal encoding makes sense.



```
# ordinal encoding: loyalty_treatment (3 categories with natural order)
# Non-Loyal < Partially Engaged < Likely Loyal

loyalty_order = {'Non-Loyal': 0, 'Partially Engaged': 1, 'Likely Loyal': 2}
df['loyalty_treatment_ord'] = df['loyalty_treatment'].map(loyalty_order)
df = df.drop(columns=['loyalty_treatment'])

print("loyalty_treatment ordinal encoding:")
print(df['loyalty_treatment_ord'].value_counts().sort_index())
```

```
loyalty_treatment ordinal encoding:
loyalty_treatment_ord
0      114692
1       96131
2       29846
Name: count, dtype: int64
```

Figure 2: loyalty_treatment Encoding code snippet

```
# ordinal encoding: scan_category (3 categories with natural order)
# Never Scanned < Intermittent Scanner < Always Scanned

scan_order = {'Never Scanned': 0, 'Intermittent Scanner': 1, 'Always Scanned': 2}
df['scan_category_ord'] = df['scan_category'].map(scan_order)
df = df.drop(columns=['scan_category'])

print("scan_category ordinal encoding:")
print(df['scan_category_ord'].value_counts().sort_index())
```

```
scan_category ordinal encoding:
scan_category_ord
0      114692
1     121942
2       4035
Name: count, dtype: int64
```

Figure 3: scan_category Encoding code snippet

- Frequency encoding for make (65 unique values) and model (892 unique values). There are way too many categories for one-hot encoding, and frequency encoding gives a reasonable numeric representation without blowing up the feature space.



```
# frequency encoding: make (65 unique -- too many for OHE)
make_freq = df['make'].value_counts(normalize=True)
df['make_freq'] = df['make'].map(make_freq)
df = df.drop(columns=['make'])

print("make frequency encoding (top 10):")
print(make_freq.head(10))
```

```
make frequency encoding (top 10):
make
HOND    0.201567
VW      0.167383
TOYO    0.159364
FORD     0.154619
NISS     0.117888
LAND     0.078639
PORS     0.070300
JAGU     0.028894
HYUN     0.002414
AUDI     0.002028
Name: proportion, dtype: float64
```

Figure 4: make Encoding code snippet

```
# frequency encoding: model (892 unique -- way too many for OHE)
model_freq = df['model'].value_counts(normalize=True)
df['model_freq'] = df['model'].map(model_freq)
df = df.drop(columns=['model'])

print(f"model unique values: {len(model_freq)}")
print(model_freq.head(10))
```

```
model unique values: 892
model
CRV      0.070545
F150     0.059239
ROGUE    0.039216
TIGU     0.036885
ESCA     0.034579
JETT     0.029663
CIVI     0.029027
RAV      0.024403
PILOT    0.023908
MACAN    0.021644
Name: proportion, dtype: float64
```

Figure 5: model Encoding code snippet

- Customer-level aggregation: We grouped by VIN and calculated total_visits (count of



transactions), avg_cost (mean cost per visit), avg_distance (mean distance traveled), and loyalty_ratio (proportion of visits with a loyalty card scan).

```
# aggregate to customer level (one row per VIN)

df_customer = df.groupby('vin').agg(
    total_visits=('loyalty_binary', 'count'), ##### consider the counting for this
    avg_cost=('cost_avg', 'mean'),
    avg_distance=('distance', 'mean'),
    avg_mileage=('mileage', 'mean'),
    vehicle_age=('vehicle_age', 'mean'),
    loyalty_ratio=('loyalty_ratio', 'first'),
    loyalty_binary=('loyalty_binary', 'first'),
    is_luxury=('is_luxury', 'max'),
    under_warranty=('under_warranty', 'max'),
    dealer_name=('dealer_name', 'first'),
    service_type_mode=('service_type', lambda x: x.mode()[0] if len(x.mode()) > 0 else x.iloc[0]),
    loyalty_treatment_ord=('loyalty_treatment_ord', 'first'),
    scan_category_ord=('scan_category_ord', 'first'),
    make_freq=('make_freq', 'first'),
    model_freq=('model_freq', 'first'),
    appointment_rate=('appointment', 'mean'),
    customer_pay_rate=('customer_pay', 'mean'),
    warranty_pay_rate=('warranty_pay', 'mean'),
).reset_index()

print(f"Transaction-level: {len(df):,} rows")
print(f"Customer-level:    {len(df_customer):,} rows")
```

```
Transaction-level: 240,669 rows
Customer-level:    53,273 rows
```

Figure 6: Customer aggregation code snippet

The final clustering feature set was: total_visits, avg_cost, avg_distance, loyalty_ratio, loyalty_treatment_ord, and scan_category_ord. We specifically chose these six features because they capture the main dimensions of customer behavior that matter for the loyalty analysis: how often they come in, how much they spend, how far they travel, and how engaged they are with the loyalty program.



```
# clustering features -- behavioral core + loyalty engagement

clust_features = [
    'total_visits',      # how often they come in
    'avg_cost',          # how much they spend per visit
    'avg_distance',      # how far they travel
    'loyalty_ratio',     # proportion of visits with loyalty card
    'loyalty_treatment_ord', # loyalty engagement tier (ordinal)
    'scan_category_ord'  # card scanning consistency (ordinal)
]

df_clust = df_customer[clust_features].copy()

# handle missing values (34 NaN in avg_cost)
df_clust = df_clust.fillna(df_clust.median())

# scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_clust)

print(f"Clustering features: {clust_features}")
print(f"Shape: {X_scaled.shape}")
print(f"NaN check: {np.isnan(X_scaled).sum()}")
```

```
Clustering features: ['total_visits', 'avg_cost', 'avg_distance', 'loyalty_ratio', 'loyalty_treatment_ord', 'scan_category_ord']
Shape: (53273, 6)
NaN check: 0
```

Figure 7: Clustering feature code snippet



5.2 Advanced Analysis:

We tested three clustering algorithms and compared them using the silhouette score, which measures how well-separated the clusters are (ranges from -1 to 1, with higher being better). Our professor set a threshold of 0.50 as the minimum acceptable score.

Algorithm	Silhouette Score	Notes
KMeans (k=5)	0.5270	Selected algorithm with the best score, clear cluster separation
GMM (k=5)	0.4438	Lower performance, softer cluster boundaries
DBSCAN (various eps)	0.484	Too many noise points, sensitive to parameters

Table 1: Algorithm Comparison

KMeans came out on top with a silhouette score of 0.527, which clears the 0.50 threshold. We picked k=5 based on both the elbow method (looking at where inertia stops dropping sharply) and the silhouette score plot (which peaks at k=5). The silhouette plot also showed that most individual data points have positive silhouette values, meaning the clusters are reasonably well-defined.

GMM scored 0.4438, which is below the threshold. DBSCAN was tricky because it is very sensitive to the eps parameter, and with our data, it either created too many noise points or lumped everything into one big cluster.

KMeans with k=5 produced five distinct customer segments. We profiled each one by looking at the average values of the key features:

Cluster	Name	% of Customers	Key Characteristics
0	Non-Loyal Regulars	~56%	Low loyalty ratio (~0), low-moderate cost, local customers
1	Engaged Power Users	~21.5%	Moderate loyalty (~0.25), slightly higher spend, consistent visitors
2	Loyal Advocates	~12.4%	High loyalty ratio (~0.73), regular visits, strong program engagement
3	High-Spend Luxury	~2.6%	Very high avg cost (~\$3,900), rare visits, almost no loyalty usage
4	Distance Travelers	~7.1%	High avg distance (~250km), above-average cost, low loyalty usage

Table 2: Cluster Profiling

Getting to a silhouette score above 0.50 was not straightforward. We went through several rounds of feature selection and encoding adjustments before landing on the final configuration. Here is a summary of what we tried:

- Initial attempt with a broader feature set including vehicle_age: silhouette dropped to



around 0.40. Vehicle age turned out to be the single most damaging feature, hurting the score by about 0.12 points. We dropped it.

- Tried frequency encoding for `loyalty_treatment` instead of ordinal: this actually inverted the semantic ordering (because the most common category got the highest value, which does not match the natural order). Switching to ordinal encoding was a key fix.
- Tested dropping `scan_category`: this brought the score below 0.50. Keeping it and encoding it ordinally was necessary to clear the threshold.
- Tested dropping `total_visits`: this actually improved the score to about 0.643, but we kept `total_visits` in the final model because visit frequency is important for business interpretability. The professor agreed with this trade-off.

The optimization process was mostly about figuring out which features help versus hurt cluster separation. We learned that more features is not always better, the wrong feature can collapse cluster boundaries. The final 6-feature set (`total_visits`, `avg_cost`, `avg_distance`, `loyalty_ratio`, `loyalty_treatment_ord`, `scan_category_ord`) at $k=5$ gave us the 0.527 score.

Beyond the main clustering, we also did a few additional analyses:

Make Specific Clustering

We repeated the clustering analysis within each of the top 7 vehicle makes (Toyota, Honda, Ford, Nissan, Volkswagen, etc. all with 1500+ customers) to check whether customer segments differ by brand. For each make we ran the same silhouette-driven k -selection ($k = 2$ to 7) to find the optimal number of clusters per brand, then compared KMeans vs. GMM.

Make-specific results from the notebook (actual output):

Make	Customers	Best k	Silhouette	Key Pattern
HOND (Honda)	10,118	5	0.526	5 segments; moderate loyalty spread across clusters
VW (Volkswagen)	8,730	2	0.531	Simple engaged vs. non-engaged split
FORD	8,602	2	0.594	Clearest two-group split; KMeans wins decisively over GMM
TOYO (Toyota)	7,920	7	0.469 (lowest)	Most complex; highest loyalty diversity; 7 sub-segments needed
NISS (Nissan)	6,526	5	0.553	Strong loyal-vs-non split; distinct power user group
LAND (Land Rover)	4,069	2	0.475	Binary engaged/non-engaged; luxury customer uniformity
PORS (Porsche)	3,176	5	0.586 (highest)	Clearest segments; luxury behavior patterns dominant; one cluster avg cost ~\$11,231

Table 3: Make-Specific Clustering Results – Best k and Silhouette per Brand

The results showed that the same general pattern appears across most makes: a large non-loyal group, a moderate engagement group, and a small highly-loyal group. However, the relative sizes differ. For example, Toyota customers had a higher proportion of loyal advocates compared to other brands, which aligns with Toyota on the Trail having the highest loyalty adoption rate among all dealers.

Dealer Performance Comparison



We aggregated customer-level data by dealership to compare performance across the seven locations. We looked at average service cost, average visits per customer, customer base size, and loyalty participation rate. The key findings were:

- Toyota on the Trail leads in loyalty engagement by a large margin (~37.6% loyalty rate), combining high volume with strong loyalty program usage.
- Porsche Centre Edmonton and Jaguar Land Rover Edmonton have the highest average service costs (\$1,200+ per visit) but the lowest loyalty adoption rates. This is the luxury paradox – premium brands have high per-customer value but barely participate in the loyalty program.
- Kentwood Ford and Norden Volkswagen also show low loyalty engagement despite having sizeable customer bases.
- Go Honda and Go Nissan South sit in the middle with moderate loyalty participation.

We also created a scatter plot of loyalty rate versus average service cost per dealer, which clearly shows the inverse relationship: dealers with higher loyalty rates tend to have lower average service costs, while luxury dealers cluster in the high-cost, low-loyalty corner.

6. Challenges Encountered:

Phase 2 came with its fair share of challenges, some on the technical side and some that were more about judgment calls where there wasn't a clear right answer.

One of the bigger headaches was encoding. In Phase 1, we used frequency encoding for `loyalty_treatment`, which made sense at the time. But when we got into clustering, we realized it was actually flipping the order where the most common category got the highest value, which doesn't match the real-world meaning of that feature at all. Switching to ordinal encoding fixed it and gave us a noticeable bump in our silhouette score.

Feature selection was another area where we kept going back and forth. We originally included `vehicle_age` because it seemed like it should matter in that older cars need more service, right? But it ended up tanking our cluster quality pretty hard. Dropping it felt counterintuitive, but the numbers made it clear that it was doing more harm than good.

Lastly, after Demo 1 our professor told us that outliers need to be deleted, not just flagged. That was a shift from what we did in Phase 1, and it changed the distribution of a few features, especially cost and distance.

7. Stakeholder Engagement:

Our main stakeholder engagement has been through the course demo sessions. During Demo 1, we presented our EDA findings and received feedback that shaped our Phase 2 approach. The clients pointed out that we should combine multiple same-day visits into single visits and that outliers need to be deleted, not just flagged.

For Demo 2, we presented the clustering results and dealer performance analysis. The feedback



was generally positive, with the professor noting that the silhouette score of 0.527 meets the threshold and the customer segments make business sense.

8. Lessons Learned:

A few things stood out from this phase:

- Feature encoding matters more than we initially thought. The difference between ordinal and frequency encoding for `loyalty_treatment` was the difference between meeting and missing the silhouette threshold. In future projects, we would spend more time upfront thinking about what each encoding implies about the data.
- Simpler models with fewer, well-chosen features can outperform complex models with many features. Dropping `total_visits` improved our silhouette score by over 0.1 points, though we chose to keep it for interpretability.
- Validation against real-world understanding is important. The clusters we found actually make intuitive sense when you think about different types of car service customers. If the segments did not make sense to a dealership manager, the analysis would not be useful no matter how good the score is.
- Team communication is key. We used Slack for daily standups and held weekly Sunday sync meetings. This kept everyone aligned, especially when we were iterating on the feature set and needed to make quick decisions.

9. Future Recommendations:

Based on our analysis, here are our recommendations for future work:

- Targeted loyalty onboarding for Cluster 0 (Non-Loyal Regulars): This is the biggest segment and they barely use the card. Even a small improvement in loyalty adoption from this group would have a large overall impact. GO Auto could try in-service reminders or first-scan incentives.
- Luxury dealer strategy: Porsche and Jaguar Land Rover customers spend a lot per visit but almost never use the loyalty card. A premium-tier loyalty program with benefits relevant to luxury vehicle owners (like complimentary detailing or loaner vehicles) might improve adoption without cheapening the brand experience.
- Learn from Toyota on the Trail: This dealership is the clear benchmark for loyalty engagement. Understanding what they do differently (staff training, customer experience, reminder systems) could help the other locations improve.
- Geographic targeting: The Distance Travelers segment shows that some customers are willing to travel far for service. This could indicate service gaps in certain areas, or it could mean specific dealers have strong reputations. GO Auto could look into whether expanding service options in underserved areas would capture more local business.
- Predictive modelling: For the final demo, we plan to add a spending prediction model (Random Forest classifier) that predicts whether a customer will be high-spend or low-spend based on their features. This would let GO Auto proactively identify and target high-value customers.

10. Impact on the Community:

The most obvious impact here is on GO Auto's day-to-day operations. If dealerships can actually



tell the difference between a loyal repeat customer and someone who just walks in once for an oil change, they can tailor how they approach each group. That means smarter use of their marketing budget and service staff instead of treating everyone the same way.

On a bigger scale, GO Auto is one of the larger employers in the Edmonton area, so anything that helps their dealerships run better has a ripple effect on local jobs and the economy around them. And honestly, if the loyalty program ends up working better because of insights like these, regular customers in Edmonton benefit too where they get a service experience that actually recognizes them for coming back.

11. Project Conclusion:

Phase 2 successfully moved our project from data exploration into actual machine learning. We built a KMeans clustering pipeline that segments GO Auto customers into five distinct groups based on their service behavior and loyalty engagement. The model achieved a silhouette score of 0.527, clearing the professor's 0.50 threshold, and the resulting segments align well with business intuition about different types of service customers.

The analysis revealed that the majority of customers (over half) barely engage with the loyalty program, which represents a significant growth opportunity. At the same time, the dealerships that do have high loyalty adoption (especially Toyota on the Trail) tend to see lower average service costs and more consistent customer relationships. For the final demo, we will bring everything together in an interactive Power BI dashboard that lets stakeholders explore these segments and dealer comparisons visually.

12. Acknowledgments:

Big thanks to our professor for the feedback throughout the project, especially after Demo 1, where the comments on outlier handling and same-day visit consolidation really changed how we approached the data going forward. We also want to thank GO Auto for giving us a real dataset and a real business problem to work on, which made this project a lot more meaningful than a textbook exercise. And of course, thanks to everyone on Team ALPACA, between the data prep, the modelling, and putting the presentation together, this was very much a group effort.

13. Appendices:

Our sprint logs and scrum notes are in the Team ALPACA Scrum Template spreadsheet, which we've included with this submission. We ran weekly sprints and tracked task assignments, progress, and what worked or didn't in retros.



Sprint 4: Submit Phase 1									
Dates: Feb 24 - Mar 2			Sprint Goal: Submit professional EDA report						
🔥 Instructions: Update task Status, dates, and notes as you work. Story points and summaries auto-calculate. See 📄 Instructions sheet .									
Task ID	User Story / Task	Assigned To	Priority	Story Points	Status	Start Date	End Date	Notes	Dependencies
S4-1	Final Phase 1 edits and submission	Manpreet	Critical	3	Complete	02/24/2026	02/27/2026	Submit by Feb 27	
S4-2	Research clustering algorithms	Ramon	High	5	Complete	02/24/2026	03/02/2026	K-means, hierarchical, DBSCAN	
S4-3	Feature engineering for clustering	Kirandeep	High	5	Complete	02/26/2026	03/02/2026	Select/create relevant features	S4-2
S4-4	Set up ML development environment	Ramon	Medium	3	Complete	02/24/2026	02/25/2026	Libraries, notebooks ready	
S4-5	Initial clustering experiments	Team	Medium	5	Complete	02/28/2026	03/02/2026	Test different approaches	S4-3, S4-4
Sprint Summary									
Total Tasks:		5							
Total Story Points:		0							
Tasks Not Started:		0							
Tasks In Progress:		0							
Tasks Complete:		5							
Sprint Completion:		100%							
Story Points Complete:		0							
Story Points Remaining:		0							
Velocity Tracking									
Planned Velocity (Story Points):		0							
Actual Velocity (Completed):		0							
Velocity Achievement:		0%							

Figure 8: Sprint 4



Sprint 5: ML Implementation										
Dates: Mar 3 - Mar 9			Sprint Goal: Implement clustering algorithms							
🔥 Instructions: Update task Status, dates, and notes as you work. Story points and summaries auto-calculate. See 📄 Instructions sheet .										
Task ID	User Story / Task	Assigned To	Priority	Story Points	Status	Start Date	End Date	Notes	Dependencies	
S5-1	Implement dealer clustering	Ramon	High	8	Not Started ▾	03/03/2026	03/05/2026	Group by loyalty adoption/performance		
S5-2	Implement customer segmentation	Ramon	High	8	Not Started ▾	03/03/2026	03/07/2026	Loyalty-driven behavior groups	S5-1	
S5-3	Validate and optimize models	Ramon	High	5	Not Started ▾	03/05/2026	03/08/2026	Silhouette score, elbow method	S5-2	
S5-4	Create model performance visualizations	Kirandeep	High	5	Not Started ▾	03/06/2026	03/09/2026	Cluster plots, metrics	S5-3	
S5-5	Interpret business insights from clusters	Manpreet	High	5	Not Started ▾	03/07/2026	03/09/2026	Connect to business questions	S5-4	
S5-6	Document model decisions and rationale	Team	Medium	3	Not Started ▾	03/08/2026	03/09/2026	Why these algorithms/parameters	S5-5	
Sprint Summary										
Total Tasks:		6								
Total Story Points:		0								
Tasks Not Started:		6								
Tasks In Progress:		0								
Tasks Complete:		0								
Sprint Completion:		0%								
Story Points Complete:		0								
Story Points Remaining:		0								
Velocity Tracking										
Planned Velocity (Story Points):		0								
Actual Velocity (Completed):		0								
Velocity Achievement:		0%								

Figure 9: Sprint 5

Sprint 6: Demo 2 Presentation										
Dates: Mar 10 - Mar 16			Sprint Goal: Present ML models and optimization							
📌 Instructions: Update task Status, dates, and notes as you work. Story points and summaries auto-calculate. See Instructions sheet .										
Task ID	User Story / Task	Assigned To	Priority	Story Points	Status	Start Date	End Date	Notes	Dependencies	
S6-1	Prepare Demo 2 presentation slides	Manpreet	High	5	Complete ▾	03/05/2026	03/09/2026	Focus on ML methodology		
S6-2	Create model visualization for demo	Kirandeep	High	5	Complete ▾	03/06/2026	03/09/2026	Interactive/clear visuals	S6-1	
S6-3	Prepare technical explanation	Ramon	High	3	Complete ▾	03/07/2026	03/09/2026	Explain algorithms clearly	S6-1	
S6-4	Rehearse Demo 2 presentation	Team	High	3	Complete ▾	03/08/2026	03/09/2026	Time and polish delivery	S6-2, S6-3	
S6-5	Demo 2 Presentation (Mar 10)	Team	Critical	8	Complete ▾	03/10/2026	03/10/2026	Present ML work to client	S6-4	
S6-6	Capture Demo 2 feedback	Manpreet	High	2	Complete ▾	03/10/2026	03/12/2026	Notes on improvements needed	S6-5	
Sprint Summary										
Total Tasks:		6								
Total Story Points:		0								
Tasks Not Started:		0								
Tasks In Progress:		0								
Tasks Complete:		6								
Sprint Completion:		100%								
Story Points Complete:		0								
Story Points Remaining:		0								
Velocity Tracking										
Planned Velocity (Story Points):		0								
Actual Velocity (Completed):		0								
Velocity Achievement:		0%								

Figure 10: Sprint 6

Sprint 7: Phase 2 Report										
Dates: Mar 17 - Mar 23			Sprint Goal: Document ML methodology and results							
🔥 Instructions: Update task Status, dates, and notes as you work. Story points and summaries auto-calculate. See Instructions sheet .										
Task ID	User Story / Task	Assigned To	Priority	Story Points	Status	Start Date	End Date	Notes	Dependencies	
S7-1	Review Phase 2 template and rubric	Team	High	1	Complete ▾	03/17/2026	03/17/2026	Understand requirements		
S7-2	Write ML methodology section	Ramon	High	8	Complete ▾	03/17/2026	03/19/2026	Detailed technical approach	S7-1	
S7-3	Create model evaluation visualizations	Kirandeep	High	5	Complete ▾	03/17/2026	03/19/2026	Performance metrics, clusters	S7-1	
S7-4	Write results and interpretation	Manpreet	High	8	Complete ▾	03/18/2026	03/19/2026	Business insights from models	S7-2, S7-3	
S7-5	Write recommendations section	Manpreet	High	5	Complete ▾	03/18/2026	03/19/2026	Actionable for GO Auto	S7-4	
S7-6	Peer review Phase 2 report	Team	High	5	Complete ▾	03/19/2026	03/20/2026	48 hours before deadline	S7-5	
S7-7	Submit Phase 2 Report (Mar 20)	Manpreet	Critical	2	Complete ▾	03/20/2026	03/20/2026	Final submission	S7-6	
Sprint Summary										
Total Tasks:		7								
Total Story Points:		0								
Tasks Not Started:		0								
Tasks In Progress:		0								
Tasks Complete:		7								
Sprint Completion:		100%								
Story Points Complete:		0								
Story Points Remaining:		0								
Velocity Tracking										
Planned Velocity (Story Points):		0								
Actual Velocity (Completed):		0								
Velocity Achievement:		0%								

Figure 11: Sprint 7



14. References

Scikit-learn Developers. (2024). *Clustering performance evaluation: Silhouette score*.
<https://scikit-learn.org/stable/modules/clustering.html#silhouette-coefficient>

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
[https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)